[name] – A Stealth Game:

By Adityan Gopalakrishnan

Table of Contents

Analysis 5

- Problem Definition and Computational Feasibility 5
- Stakeholders 7
- Research on solutions to similar problems 8
- Essential features of the proposed solution 22
- Limitations of the proposed solution 29
- Solution Requirements: 31
- Success Criteria 38
- Approach to Design and Development 51

Design 51

- Top Level Decomposition 51
- Game Decomposition 55
- UI Decomposition 61
- Class Diagrams 65

Development 65

Testing 66

Evaluation 67

Analysis

Problem Definition and Computational Feasibility

The problem involves designing an interactive system in which a user navigates a dynamic environment while avoiding detection by autonomous agents that react to movement and environmental factors in real time. The system must continuously respond to changes in the user's behaviour and position while maintaining consistent and intuitive agent responses. The proposed solution is a two-dimensional, top-down interactive stealth system in which the user controls a character navigating through a structured environment composed of interconnected areas. The system incorporates autonomous agents that exhibit adaptive behaviour in response to the user's actions and environmental conditions.

The solution addresses the problem by modelling stealth interactions using quantifiable factors such as visibility, audibility, and proximity, allowing the system to evaluate detection dynamically in real time. Autonomous agents respond to these factors through structured, state-based behaviour and controlled movement, creating consistent and intuitive interactions. The environment is designed to support these mechanics through the use of obstacles, cover, and controlled pathways, enabling meaningful stealth challenges.

Why this problem is amenable to a computational approach:

The problem is amenable to a computational approach because it can be decomposed into a set of smaller independent systems each governed by clear rules and measurable inputs. Movement of the user-controlled character, behaviour of autonomous agents and user feedback can all be implemented as discrete computational components that interact in real time. This decomposition allows complex stealth behaviour to emerge from the interaction of simpler rule-based systems, making the problem well-suited to algorithmic implementation.

Features that make the problem solvable by a computational approach

Stealth interactions can be represented using measurable parameters such as visibility, audibility, and proximity, which can be expressed numerically and updated dynamically as entities move within the environment. These parameters allow algorithms to evaluate detection likelihood in a consistent and repeatable manner, enabling realistic stealth interactions.

The movement and behaviour of autonomous agents are amenable to computation through algorithmic pathfinding and state-based behaviour models. Agent behaviour can be categorised into discrete states, such as patrolling, suspicious, and alerted, with transitions governed by defined logical conditions. This allows complex behaviour to emerge from structured, rule-based systems.

The problem also involves the management of structured data that must persist and update throughout execution. This includes tracking the current state of the game, storing user-defined settings, and maintaining save data that represents progress. These forms of data require organised storage, retrieval, and validation, which are naturally suited to computational methods. A computational approach allows this data to be represented using structured data models and updated efficiently in response to user input and system events. Without computation, managing consistent state across gameplay sessions and system transitions would be impractical.

The behaviour of both the user-controlled character and autonomous agents can be represented using state-based models, where entities exist in clearly defined states and transition between them in response to events. This allows complex behaviour to be constructed from a set of controlled, rule-based conditions. State-based behaviour is particularly suitable for a computational solution, as it enables predictable transitions, simplifies decision-making logic, and ensures consistent responses to user actions. This approach supports real-time evaluation of behaviour while remaining computationally efficient.

Sound and vision modelling can be implemented using mathematical and geometric principles. Line-of-sight calculations determine visibility based on relative position, direction, and obstacles, while audibility can be modelled using distance-based calculations and movement states. These computations allow agents to react accurately to player actions.

Furthermore, the use of a two-dimensional top-down environment reduces computational complexity by limiting the number of spatial dimensions that must be processed. This allows spatial calculations such as visibility and sound propagation to be performed efficiently, supporting real-time updates while maintaining responsive system behaviour.

Stakeholders

Stakeholder 1: Ying (Primary End User):

Ying is an 18-year-old student who enjoys competitive and action-oriented games. As a player, he values systems that reward skillful decision-making and allow players to choose between riskier actions or safer approaches.

As a stakeholder, Ying's expectations emphasise the importance of clear trade-offs between speed and safety in gameplay. This directly informs the design of the movement and noise systems, where running allows rapid repositioning at the cost of generating high noise, while sneaking enables safer traversal at reduced speed. This ensures that player success depends on informed decision-making rather than reflexes alone.

Stakeholder 2: Shlok (Primary End User):

Shlok is a 17-year-old student who enjoys strategy-focused games that prioritise planning and positioning. He prefers gameplay that challenges players to analyse situations and adapt rather than relying on constant action.

As a stakeholder, Shlok's needs are addressed through the use of a state-based enemy AI system. Enemies transition between patrol, suspicious, searching, and chasing states based on player actions, creating predictable but flexible behaviour. This allows players to plan their movement and interactions strategically, reinforcing stealth as the optimal approach.

Stakeholder 3: Lewis (Primary End User)

Lewis is an 18-year-old student with extensive experience in 2D top-down games. He values clear spatial reasoning, predictable movement, and consistent environmental feedback.

As a stakeholder, Lewis's expectations influence the design of the movement and environment systems. The use of a top-down perspective with 360-degree movement ensures precise positioning, while collision with walls and obstacles enforces clear spatial boundaries. This allows players to accurately judge distances and positioning, which is essential for stealth-based navigation.

Stakeholder 4: Dylan (Primary End User)

Dylan is an 18-year-old student who frequently plays stealth-focused games and values fairness and clarity in gameplay feedback.

Dylan's needs are addressed through the design of the user interface and feedback systems. The HUD communicates critical information such as movement mode, enemy awareness states, and cooldowns. This ensures that stealth outcomes are based on player decisions rather than hidden or ambiguous mechanics, maintaining fairness and readability.

Research on solutions to similar problems

Metal Gear Solid Delta – Snake Eater (2025)

Metal Gear Solid Delta: Snake Eater is a 2025 action-adventure **stealth** game developed and published by Konami.

Line of sight detection system:

In Metal Gear Solid (MGS), the line-of-sight system is central to stealth gameplay.
It determines whether enemies can see the player, which influences enemy detection and alertness. The design relies on clear geometric and logical rules that simulate human vision computationally.

Each enemy has a vision cone — a triangular region projecting forward from their position, representing their field of view (FOV). Typically, this covers around 60–90° in front of the enemy and extends a certain distance depending on the difficulty or environment (for example, longer in bright areas, shorter in darkness or fog).

The game uses raycasting to determine whether the player is visible. If a straight line from the enemy's eyes to the player is not blocked by any wall, object, or environmental obstruction (e.g. darkness), the player is considered visible.
In 3D, this involves complex mesh intersection checks to determine whether the player's model lies within the planes of the enemy's vision. In 2D, this simplifies to polygon intersection tests, making it computationally lighter and ideal for a top-down perspective.

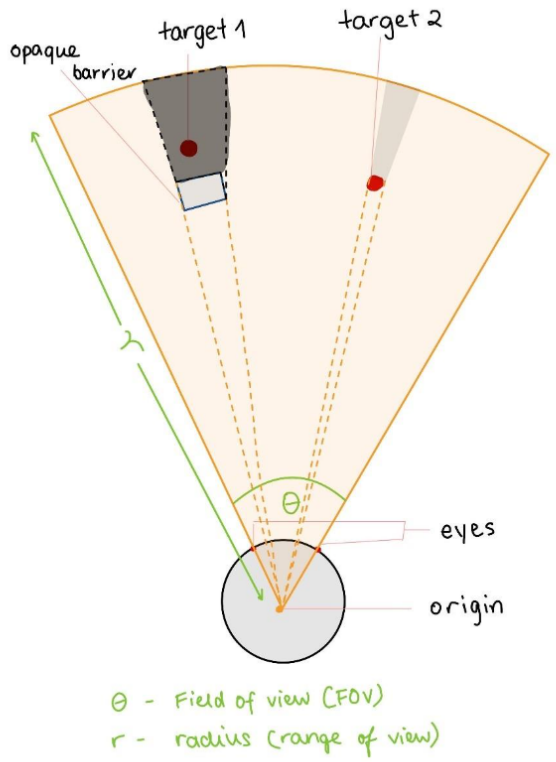
How this may be implemented:

A 2D version of this system can be implemented using sector-based raycasting, following the same logic as MGS but simplified for a top-down environment.

Each enemy is assigned a direction, and a FOV angle. A cone shape projects forward from the enemy to represent their vision

Multiple rays are cast within the FOV to check for intersections with walls. If at least one ray reaches the player unobstructed, visibility is confirmed.

The diagram below shows how sector-based raycasting would work:



In the diagram, the black circle represents the enemy. At the centre of the circle is the origin of the cone, which is where computationally the lines are drawn from the enemy's hitbox. Two lines are draw from the origin to create a sector of length r, the range of vision. The angle the two lines create is the FOV, θ, in degrees. The diagram also shows two targets, where one is hidden behind an opaque barrier, and two is within line of sight.

The diagram shows how the line-of-sight system decides whether targets are visible to the enemy. It does this by casting multiple rays along the sector, and noting down the first opaque object this ray comes across. If this opaque object is a target, then the target is detected by the enemy.

However, the accuracy of this system depends heavily on the density and distribution of the rays. If only a few rays are cast within the vision cone, small gaps in coverage can cause situations where the player is partially visible but not detected. Conversely, if too many rays are used, the system may detect the player when only a tiny portion of the sprite is exposed, leading to unfair or frustrating gameplay.

Design Implications:

This system's strength lies in its predictability and precision, because players can learn how far they can push visibility without being caught. This builds tension and rewards spatial awareness, two key pillars of stealth gameplay found in Metal Gear Solid.

The Metal Gear Solid line-of-sight system is suitable for the proposed solution because it can be cleanly abstracted into a 2D top-down context using angular sectors to represent enemy vision. This approach allows enemy detection to be determined through simple geometric checks based on position and direction, while still supporting occlusion by walls and obstacles. By using a sector-based line-of-sight model rather than complex lighting or visibility calculations, the system remains computationally efficient, predictable, and fair to the player, making it well-suited to a real-time stealth game implemented in a 2D environment. Additionally, to ensure this system remains fair and balanced, several measures can be introduced:

- Increase the number of rays slightly near the centre of the cone (where enemies focus most) and reduce them at the edges. This reflects realistic peripheral vision and optimises performance.
- Require multiple rays to simultaneously detect the player before triggering an alert. This prevents accidental detections from minimal exposure.

- Introduce a short confirmation delay between initial visibility and full detection, giving the player time to react or retreat.

Enemy Search and Investigative Behaviour:

In Metal Gear Solid (MGS), enemy search and investigation behaviour is a defining feature of the stealth experience. It transforms detection from a simple binary event into a dynamic process that adds tension, realism, and player agency. Rather than instantly attacking when alerted, enemies in MGS follow a structured sequence of behavioural states that simulate human reasoning — observation, investigation, pursuit, and return to patrol.

Behavioural States in MGS:

Normal/Patrol State - Enemies follow predetermined patrol routes within a set radius or along fixed paths. They occasionally pause to look around, creating natural observation windows and preventing predictability.

Suspicious/Investigative State - This is triggered by partial detection cues such as hearing a noise, noticing movement, or briefly spotting the player at the edge of their vision cone. The enemy moves cautiously toward the source of the disturbance, often calling out or changing stance. The enemy's AI sets a temporary target location (the point of interest) and navigates there using simple pathfinding.

Alert/Combat State - Activated when the player is fully visible or confirmed as hostile. The enemy communicates with others via radio, and the area is marked as dangerous. Enemies aggressively pursue and engage until they lose sight of the player for a set duration.

Caution/Search State: - Once the player evades direct line of sight, enemies do not immediately return to normal behaviour. They instead search the nearby area, using a grid-based or radius-based sweep pattern centred on the player's last known position. After a fixed time or failed search, enemies gradually return to their patrol routes.

The behavioural state system used in Metal Gear Solid demonstrates a highly sophisticated approach to enemy AI, with multiple layers including patrol, suspicion, caution/search, and alert/combat states. This system allows for nuanced responses to

player actions, incorporating pauses, stances, audio calls, and coordinated communication. While effective in a 3D, resource-rich environment, many of these mechanics are overly complex for a 2D top-down stealth game implemented in Pygame, where computational resources and visual abstraction are more limited. For this reason, a simplified state-based system is proposed, consisting of four core states: Patrol, Suspicious, Searching, and Chasing. In this model, enemies follow set patrol routes, accumulate suspicion through quantifiable inputs such as noise or dead bodies, search areas based on last-known player positions, and chase the player upon confirmed visual detection.

Design Implications:
Research into enemy behaviour in stealth games demonstrates that detection should be a gradual process rather than a binary outcome. Therefore, enemy AI in the proposed solution will be structured using a finite state machine, with distinct states such as patrol, investigate, search, and chase. This approach allows enemies to react proportionally to partial information, such as sounds or brief visual contact, increasing tension without overwhelming the player. Limiting search duration and patrol boundaries will ensure that behaviour remains believable while preventing excessive difficulty.

Sound and Noise Mechanics:
The sound system in MGS simulates auditory perception, where both the player and enemies can create or respond to noise. This adds a second sensory dimension to stealth — complementing vision-based detection with hearing-based investigation.

Sound Detection in MGS

Enemies in *Metal Gear Solid* respond to a wide range of auditory cues, including:

- Walking or running creates noise depending on movement speed and surface type.
- Knocking on walls, gunfire, explosions, or breaking objects trigger specific AI responses.
- Tools such as empty magazines or thrown objects can be deliberately used to lure guards away.

When a sound is produced, the AI calculates whether it falls within the enemy's *hearing radius*. If it does, the guard transitions into an *investigate* or *caution* state, setting the sound's origin as a temporary target location.

While highly effective in a 3D stealth environment, MGS's auditory system is complex and computationally intensive. It requires real-time calculation of sound propagation, distance attenuation, and AI response prioritisation. The richness of these mechanics provides deep emergent gameplay but may be impractical in a simpler 2D top-down environment implemented in Pygame, particularly given resource constraints and the need for clear, readable feedback to the player.

To retain the strategic depth of auditory mechanics while remaining feasible, the proposed solution implements a simplified noise system. Each player action (sneaking, walking, running) generates a quantifiable noise value. Noise propagates through the environment with intensity decreasing according to distance, using an inverse-square relationship to simulate natural attenuation. Enemies detect noise if its intensity exceeds a threshold and use it to adjust their suspicion score, which may trigger a transition from Patrol to Suspicious or Searching states. Unlike MGS, the system abstracts surface types and complex object sounds, focusing instead on clarity and predictability. This approach preserves the core tactical dimension of sound-based stealth while being computationally simple and transparent to the player.

Design Implications:
The research indicates that sound is most effective in stealth gameplay when it encourages investigation rather than immediate confrontation. Consequently, sound events in the proposed solution will generate investigation behaviour rather than instant alerts. Sound detection will be based on distance and environmental obstruction, allowing players to intentionally manipulate enemy movement through distractions. This design choice reinforces strategic play while maintaining fairness and computational efficiency.

Player Movement and Stance System:

In Metal Gear Solid, the player's movement and stance system is one of the most important gameplay elements defining stealth effectiveness. Different movement speeds and body positions (standing, crouching, crawling) influence how visible and audible the player is, directly linking player choice to enemy perception. This system encourages risk–reward decision-making — faster movement allows quicker traversal but increases noise and visibility, while slower movement provides stealth advantages.

Movement and Stance in MGS

The stance system offers fine-grained control over player posture and pace:

- **Standing:** Fastest movement, highest visibility and noise generation.
- **Crouching:** Moderate speed, reduced visibility and sound.
- **Crawling:** Minimal speed, near-silent movement, lowest visibility.

These stances also affect how the player interacts with cover and environment: crawling under vents, leaning around corners, or taking advantage of terrain elevation. The system merges animation, physics, and AI visibility checks to deliver an intuitive stealth experience.

Limitations in a 2D Top-Down Context

In a 2D top-down game, physical nuances of movement cannot be represented visually or spatially with full fidelity.

Key limitations include:

- The 2D perspective makes it harder to convey the difference between running, walking, or sneaking visually.
- Players rely on consistent cues or animations to perceive movement speed, rather than posture or height.
- Complex 3D movement behaviours, like sprinting past obstacles or carefully hugging corners, must be abstracted to tile- or zone-based systems.
- Movement speed affects detection primarily through noise mechanics rather than visual cues.

Overcoming Limitations:

Each movement speed can be represented through its effect on noise, rather than animation:

- Running → High speed, generates a large noise radius.
- Walking → Moderate speed, generates moderate noise.
- Sneaking → Low speed, generates minimal noise.

This preserves tactical decision-making, allowing players to balance speed against the risk of detection.

Simple visual cues can reinforce movement mode and provide player awareness:

- Small icons above the player to indicate sneaking, walking, or running.
- Slight sprite tinting or shading to reflect cautious versus fast movement.

Movement speed interacts with environmental zones to influence stealth outcomes:

- Low cover tiles reduce detection when running or walking.
- Noise-dampening zones (e.g., thick carpet or grass) reduce sound propagation for all movement speeds, encouraging strategic positioning.

Design Implications

Due to the limitations of a 2D top-down perspective, complex posture-based stealth mechanics must be abstracted rather than visually replicated. The proposed solution will represent player stance through data-driven modifiers that affect sound output, rather than detailed animations. This preserves the strategic impact of stance-based decision making while remaining suitable for a 2D environment. Clear visual indicators will be used to communicate stance changes to ensure player understanding and responsiveness.

Hades (2020)

Hades is a 2020 roguelike action role-playing game developed and published by Supergiant Games.

Core movement:

One of the reasons *Hades* feels fluid and responsive is its top-down movement system. The game uses continuous, velocity-based movement rather than grid-based steps, allowing precise control over direction and speed. This smooth movement gives players fine-grained control, which is particularly important in stealth gameplay where careful positioning and timing are critical.

The player can move in eight directions - up, down, left, right, and diagonally - using either keyboard input or an analogue stick. Unlike grid-based movement, *Hades* implements a continuous velocity-based system, giving fine-grained control that feels responsive and natural.

In a 2D stealth context, this smooth control is crucial for:

- Precise positioning when sneaking around enemies' vision cones.
- Reactive evasion when caught or when moving between cover points.

- More realistic navigation, avoiding rigid grid patterns that would make stealth predictable.

Hades implements acceleration and deceleration curves — meaning the player character does not instantly start or stop moving. This gives weight to movement and enhances realism.

Design Implications:

This research into Hades' continuous eight-directional movement and momentum handling demonstrates that subtle control and realistic acceleration enhance player agency and responsiveness. In the context of a 2D stealth game, integrating smoother movement with better control complements the variety in movement types and the navigation of enemy-dense territory without detection, making stealth gameplay more meaningful.

Perspective System:

Hades employs a fixed, top-down isometric camera that keeps the player at the centre of the screen throughout gameplay. This camera design is fundamental to the game's readability — it ensures that both the player and enemies are always visible and that the player can clearly perceive distances, attack ranges, and safe zones.

Unlike dynamic third-person cameras, which constantly shift angles, Hades relies on a consistent orthographic projection. This eliminates perspective distortion, making object positions and hitboxes easy to interpret — an especially important feature for precision-based gameplay.

The camera in *Hades* is largely fixed but subtly dynamic in context:

- During normal exploration, the camera maintains a locked zoom and centred framing, giving the player a stable reference point.
- During combat or boss fights, it zooms out slightly to show more of the arena, allowing players to react to multiple threats simultaneously.
- The camera also performs minor smoothing or panning when the player dashes rapidly across the map, preventing sudden, disorienting transitions.

This subtle responsiveness makes movement feel fluid while keeping the player grounded within the game world — a principle that can translate effectively into stealth gameplay.

Design Implications:
This research highlights that a stable, consistent viewpoint is crucial for player readability and spatial awareness in stealth gameplay. For this game, a basic fixed top-down perspective will be implemented, keeping the player and enemies clearly visible at all times. Zooming will not be used, as the levels are small enough to fit the screen and maintaining a fixed scale ensures that distances and line-of-sight are easy to judge. This perspective supports precise movement, tactical planning, and clear understanding of enemy positions, aligning with the game's core stealth mechanics.

Environmental and Visual Design:
In Hades, environmental design plays a crucial role in ensuring gameplay clarity and spatial awareness within its top-down perspective. The game is visually plenteous yet it remains readable; players can instantly distinguish walls, accessible ground, hazards and interactable element amidst chaotic combat. For a stealth game, this clarity is essential to allow players to plan routes, identify hiding spots and distinguish between safe areas and dangerous areas.

Although Hades is a 2D top-down game, it gives the illusion of depth through layering and perspective cues.

- The foreground and background layers overlap subtly to convey distance.
- Shadows under objects and characters help "anchor" them to the floor.

In the image above, the foreground and background are two separate layers, where the foreground is the playable area, and the background is the inaccessible area. This is shown through the illusion of dimension, where solid lines and changes to colour allow for a 3D space. This approach maintains the illusion of a 3D space without using complex geometry.

A layered environment in a stealth context can be used to:

- Indicate cover height — e.g., tall walls fully block vision, while low barriers only obscure crouched movement.
- Improve player-enemy spatial reasoning, ensuring the player always understands whether they are hidden or exposed.
- Support environmental storytelling, with layers used to subtly suggest tactical spaces, like shadowed corners or raised platforms.

Every element in Hades serves a visual function; walkable areas are lit with consistent textures, while obstacles use heavier contrast or darker tones. Borders are often softened or highlighted to visually guide player movement.

Chiaroscuro is an art technique used in Hades that uses strong contrasts between light and dark to dramatically define the forms and the mood of a composition. Originating in the Italian Renaissance, the word itself comes from "chiaro" (bright or clear) and "scuro" (dark or obscure), referring to the interplay between illumination and shadow within an artwork. The technique is used to create a sense of three-dimensionality and volume, often resulting in visual drama and emotional intensity.

Translating the technique to video games like Hades, chiaroscuro manifests in dungeon arenas through explicit design choices:

- Edges of battle rooms are submerged in darkness while the central areas remain brightly lit, juxtaposing safety and danger, or clarity and uncertainty.
- Saturated colors (reds for Asphodel, blues for Elysium) are paired with extreme shadow, forming vivid silhouettes of characters and interactive elements against near-black backgrounds. This not only adds a sense of mystery but ensures visual clarity in the heat of gameplay.
- The effect heightens energy and atmosphere, invoking the emotive weight of classic paintings where figures emerge from the shadows, underscoring narrative tension within the game.

Design Implications:

This research highlights that environment and visual design is crucial for player readability and spatial awareness in stealth gameplay. For the proposed game, contrasting colours and simple layering will be used to clearly differentiate walls, obstacles, and walkable areas, ensuring players can quickly interpret the environment. Foreground elements, such as obstacles and cover, will be visually distinct from the background to provide clear tactical information, while subtle shadowing will anchor characters and objects to the floor. By abstracting principles from the Chiaroscuro technique in *Hades*, the game can communicate safe zones, enemy positions, and cover opportunities without complex 3D geometry, supporting strategic planning, precise movement, and fair stealth interactions within a 2D top-down perspective.

Mark of the Ninja (2012)

The Use of Light and Shadow

The game utilises stark visual contrast – characters in darkness are rendered as black silhouettes with white outlines, while those in light are shown in full colour.
This immediately communicates visibility states to the player, removing confusion and supporting instant readability.

Player shown in darkness – silhouette outlined in white

Player shown in full colour – exposed in light (right)

The same is said for enemies, where enemies in the dark are silhouettes in white outlines. The smart use of silhouettes to represent the state of darkness allows for visibility and clarity in gameplay whilst being logically sound.

Lamps and flashlights cast hard-edged circles or cones, defining safe and dangerous zones without ambiguity. The clear separation allows for tactical planning and instant recognition of enemy sightlines. This is shown from the light in the image above.

Sound is visualized by expanding circles, exactly showing how far noise travels. Enemy vision is depicted as coloured arcs or cones so the player is always aware of what enemies can detect and when a mistake is made.

Players can extinguish light sources and hide in background objects, thus manipulating the light and shadow environment as active gameplay mechanics.

The game likely implements a “light mask” overlay using shaders or matte layers; a simple, computationally efficient way to cut out areas of the scene for “light” or “shadow.” Sprite coloration (full colour or silhouette) is dynamically toggled based on these mask intersections.

Enemy vision cones are implemented using basic geometric calculations (angle, distance, occlusion via raycasting), which is straightforward in top-down or platformer engines, and lightweight for performance.

Entry into lit or shadow zones, as well as sound circles intersecting enemies, can be handled with standard 2D collision detection, no complex 3D lighting

calculations required, keeping gameplay responsive and accessible for indie development.

The character sprite swaps between “silhouette” and “colour” states based on their overlap with shadow zones; all visual feedback is driven by in-game metadata, not manual art asset updates

Why this works in stealth:

- These systems provide immediate visual validation of player actions; vital in stealth games where ambiguity can frustrate or undermine tactical play.
- All visual feedback mechanisms rely on basic 2D math; shaders, area checks, sprite toggles, making them perfectly suited to solo or small-team projects and technology stacks for top-down games.
- By allowing players to manipulate light and shadow and instantly see the effect, you empower them to experiment, make informed choices, and strategize under clear rules which is a direct reflection of Mark of the Ninja’s critical success in the genre.
- Immediate feedback makes testing and iteration far easier; errors are visible, and the stealth experience is rewarding at all development stages.

Design Implications:
This research demonstrates that immediate visual feedback is essential for effective stealth gameplay. However, implementing dynamic lighting and shadow-based detection adds considerable complexity, including constantly updating enemy visibility based on changing light conditions and shadowed areas. For a 2D top-down game, this would be difficult to balance and computationally expensive and could make player decisions less predictable. Therefore, this game will use a simplified line-of-sight system, ensuring clear, consistent feedback while maintaining tactical depth and allowing players to make informed stealth choices without unnecessary complexity.

Essential features of the proposed solution

[name and describe these features]

[explain why you chose these features]

Player Movement System

Feature	Description	Justification
360-degree movement	The player is able to move freely in any direction across a full 360 degrees, rather than being restricted to fixed horizontal, vertical, or diagonal directions. Movement direction is determined by mouse position and keyboard input, allowing smooth and precise navigation around obstacles, enemies and environmental features	Allowing 360-degree movement improves player control and precision when navigating complex environments. In a stealth-based game, fine positioning is essential for avoiding enemy line-of-sight, staying within shadows, and moving quietly around obstacles. Smooth directional control allows players to make small, deliberate adjustments to their position, reducing frustration and increasing the sense of control and immersion. This approach supports strategic movement and careful planning, which aligns with the expectations of Stakeholder 3, who values clarity, responsiveness, and precise control in top-down, tile-based or grid-influenced environments.
Varied Movement Speeds	The player can move at different speeds, allowing them to balance movement efficiently with risk of detection. Movement speed directly affects the amount of noise generated, encouraging strategic decision-making when navigating enemy controlled areas.	Multiple movement speeds are used to allow the player to control the level of noise they generate, creating meaningful trade-offs between speed and stealth. This supports different gameplay scenarios, such as moving silently near enemies or moving quickly through low-risk areas, and integrates directly with the enemy detection system. This aligns with the expectations of Stakeholder 1 as well.

Collisions with walls and obstacles	The player cannot move through walls or obstacles and is stopped when a collision with the environment occurs	Collision handling ensures the player remains within the defined level boundaries and interacts realistically with the environment. This is essential for maintaining clear stealth routes and preventing unintended shortcuts that could undermine gameplay design and balance.
Movement when carrying bodies	When the player is carrying a body, their movement speed is restricted and running is disabled until the body is dropped	Restricting movement while carrying bodies introduces a risk-reward trade of, preventing the player from neutralising enemies without consequence. This encourages careful planning of attacks rather than aggression.

Player Interaction System

Feature	Description	Justification
Attack	When in proximity with an enemy, this interaction ability kills the enemy to a dead state. Can only be performed when out of sight from target enemy.	This basic short-range attack allows the player to fight back, although the lack of range makes combat unappealing, which accentuates the importance of strategy rather than aggression, ensuring that stealth is the main focus of the game, rather than combat.
Body carrying and dropping	When in proximity with a dead enemy, the player can carry the enemy and drop them elsewhere.	This interaction is important as it provides counterplay to the enemy detection system. Hiding dead bodies may stop enemies from finding them and starting the room-wide search. To balance this ability, noise is made when dropping the body, which can be detected by nearby enemies who may come to the place where the dead enemy is dropped.

Player Visibility	The player can only see what is around them, emulating real visibility	This is important to balance the game, the player should have to peek angles in order to get information, rather than the whole map be visible, otherwise it would be too easy.

Enemy AI and Detection System

Feature	Description	Justification
Patrol Routes	These are pre-defined routes that enemies follow when in the patrol state, at a walking speed.	Patrol routes are necessary to act as a default state for enemies when they lose suspicion. Being predefined allows them to be predictable, letting players plan routes and timings dependent on the patrols. They also act as the routes for the room-wide search.
State-based enemy system	This is a model of the states that the enemy can be in during a stealth scenario; patrol, suspicious, search and chase states. These states are activated/deactivated dependent on the player's action and noise. There is another permanent state, dead which follow from player attacks.	This is necessary to allow for transitions in enemy logic after a change in enemy inputs to simulate detection. This aligns with the expectations of Stakeholder 2
Noise-based detection	This is an input in the state-based enemy system where the enemy keeps track of noise. There are two levels of perceivable noise, noticeable and detectable. Noticeable noise is only heard by the enemy, detectable noise is heard and its source can	This allows the noise of player actions to be quantified and detected by enemies dependent on distance and intensity, acting as inputs for detection, meaning the player's position can be tracked by enemies, allowing for engaging

	be determined by the enemy. The input noises are generated by actions that the player performs.	stealth gameplay, and creating the need for the player to control noise output. This aligns with the expectations of Stakeholders 1 and 2.
Distance-based noise fall-off	To realistically portray noise, noise created has an intensity associated with it, and it travels as an expanding circle around the source of noise. As the radius of the circle grows, the intensity that is perceived reduces. When an enemy hears a noise, the perceived intensity of the noise is calculated by its distance from the source and the original intensity of the noise.	This simulates the effect of noise falloff due to air resistance in real life, and justifies the need for intensity thresholds in the noise-based detection, as not all intensities of noise can be heard. This aligns with the expectations of Stakeholders 1 and 2.
Line-of-sight based visibility	This is an input in the state-based enemy system where the enemy keeps track of the field of vision in front of them, and checks for the player within this region.	This simulates visual detection, as it checks for an overlap between entities and the vision cone of enemies, feeding back to the detection system.
Suspicion score accumulation and decay	Enemies gradually accumulate suspicion when they detect potential signs of the player, such as noises or bodies. Suspicion determines whether an enemy investigates further, enabling a smooth transition between patrol and search behaviours.	This system provides predictable and consistent enemy responses to player actions, encouraging strategic planning and rewarding careful stealth. By gradually increasing suspicion, the game allows players to anticipate enemy reactions rather than facing sudden or unfair detection. This aligns with the expectations of Stakeholder 2.

Game Environment

Features	Description	Justification
Walls	Walls act as impassable barriers that structure the level. They constrain movement, create strategic pathways, and allow the design of stealth encounters by controlling line-of-sight and player navigation.	Walls define navigable space, by marking boundaries, shaping the player's movement and the structure of the level. Without walls, players could move anywhere, which removes strategy and stealth opportunities. By blocking movement, walls create choke points, pathways and areas for hiding or ambushing, which are essential for stealth mechanics.
Obstacles	Obstacles act as impassable objects which block movement and vision, however they are placed to cut vision, leading to their use in hiding out of sight.	Obstacles break the monotony of walls, because they can exist in open areas. They create hiding spots for the player but can also slow movement. This is used to balance the game, as without any hiding spots, there is no counterplay to being detected, but timing.
Doorframes	Doorframes connect different sections of the level, signalling entry/exit points without creating invisible boundaries	They allow controlled movement between areas which can be used for creating tension and engaging gameplay (a guard may patrol near the doorway). This element is also used as a key waypoint for enemy pathfinding.
Waypoints	These are intrinsic, invisible points in the level which are used for enemy pathfinding and setting patrol routes.	Waypoints allow for the omission of a tile-based system, which can be expensive when pathfinding, and permits 360-degree enemy movement as well, making

		it follow the same logic as the player, making the game's movement uniform.
Hiding Spots	These are visible points within the level which the player can interact with, making them hidden.	Hiding spots support stealth-focused gameplay by allowing the player to avoid enemy detection without direct combat. They encourage careful positioning and planning, reinforcing the core stealth mechanics of the game. This reduces reliance on combat and increases player choice in how challenges are approached.

User Interface

Features	Description	Justification
Start Menu	The start menu is displayed when the game is launched. It allows the player to play the game, access the settings menu, or exit the application	The start menu provides a clear entry point into the game. It ensures that the player understands the available options before gameplay begins. It also prevents the game from starting unexpectedly and allows system resources to be prepared before entering the play state.
Settings Menu	The settings menu allows the player to adjust different game options such as display settings, controls, etc.	Providing a settings menu improves accessibility and user experience by allowing players to customise the game to

		their preferences. This reduces frustration caused by unsuitable default settings and ensures the game can be used comfortably on different systems
Pause Menu	The pause menu appears as an overlay while gameplay is frozen, with the current game frame visible in the background	Using a pop-up pause menu preserves the player's spatial awareness and context within the level. Freezing the game state prevents unfair gameplay consequences while allowing the player to adjust settings or resume play without reloading the level.
Level completion	A pop-up overlay is displayed when the player successfully completes a level, with gameplay frozen in the background.	Displaying the completion screen as an overlay reinforces the link between the player's actions and the outcome. It avoids unnecessary scene changes, maintains immersion, and allows smooth transitions to the next level or menu.
Level Failed Screen	A pop-up overlay appears when the player fails a level, showing failure feedback while the game remains frozen in the background.	A pop-up overlay appears when the player fails a level, showing failure feedback while the game remains frozen in the background.

Limitations of the proposed solution

Complexity of Enemy AI Behaviour:

- Enemy AI may be too predictable as it is rule-based, players can find patterns and could exploit vulnerabilities too easily, leading to balance issues within the gameplay.
- Advanced behaviours such as coordinated attacks and searches are difficult to implement and outside the scope of this project.

2D Top Down Format:

- A 2D top-down format reduces perspective depth and verticality, meaning certain stealth mechanics cannot be implemented well, e.g. crouching behind walls.
- Visual feedback may be hard to interpret due to the constraint in verticality.

Simplified Modelling of Sound and Vision:

- Enemy perception uses only line of sight checks, independent of distance or light levels, reducing realism of stealth scenarios.
- Noise is modelled as an inverse square fall off with regard to distance, and doesn't take account of walls/obstacles, therefore may not be realistic
- Cannot simulate nuances in hearing and vision, e.g. peripheral vision or echoes, etc.

Performance Limitations on Lower-End Hardware:

- Lower-End Hardware may experience frame drops due to processing of Enemy AI, level handling, multiple enemies, etc.

Lack of Animation/Art/SFX:

- Visual and auditory feedback for actions is minimal, and game looks plain and inaesthetic
- The user may find gameplay less engaging or harder to interpret

Level Design Limitations:

- Levels cannot be too large as it will affect performance negatively.

Limited save game features:

- Players can only save progress by completing a level and unlocking the subsequent level.
- Progress midway through a level will not be saved.

Auditory Feedback Limitations:

- Due to the scope of the project, auditory feedback will be limited; only essential gameplay feedback sounds will be used
- Users may find gameplay confusing or be unaware of enemy awareness and location.

Solution Requirements:

Functional Requirements

ID	Requirement Description	Priority
FR1	The player shall be able to move in all directions at different speeds	High
FR2	The player's movement speed shall be frame-rate independent.	High
FR3	The player shall rotate to face the direction of movement.	Medium
FR4	The player shall be able to interact with objects such as doors or hiding spots.	High
FR5	The player shall be able to perform stealth actions that reduce detection.	High
FR6	The player shall be able to perform noise-producing actions.	Medium
FR7	The player shall not be able to move through solid objects.	High
FR8	The player shall be unable to perform actions while in restricted states.	Medium
FR9	The player shall have multiple states	High
FR10	Player state shall affect enemy behaviour.	High
FR11	Enemies shall detect noise within a defined radius.	High

FR12	Noise detection radius shall vary by action.	Medium
FR13	Enemies shall detect the player via line of sight.	High
FR14	Line of sight shall be blocked by obstacles.	High
FR15	Enemies shall patrol predefined routes.	Medium
FR16	Enemies shall search last known player location.	High
FR17	Enemies shall return to patrol after search timeout.	Medium
FR18	Enemies shall chase the player when visible.	High
FR19	Enemies shall attack within a defined range.	High
FR20	Enemy attacks shall kill the player.	High
FR21	Enemies shall transition between AI states.	High
FR22	The game shall support multiple levels.	Medium
FR23	Levels shall load from external files.	High
FR24	Level data shall store enemies, patrols, and obstacles.	High
FR25	Levels shall have active, completed, and failed states.	Medium
FR26	The game shall enter play state during gameplay.	High
FR27	The game shall pause when pause input is pressed.	High
FR28	The game shall enter completion state on success.	Medium
FR29	The game shall enter dead state when player is killed.	High
FR30	The start menu shall allow starting or exiting the game.	High
FR31	The pause menu shall allow resuming or quitting.	High
FR32	The settings menu shall allow modifying settings.	Low
FR33	UI shall update based on game state.	High

Non-Functional Requirements

ID	Requirement Description	Priority
NFR1	The game shall run at a minimum of 60 FPS.	High
NFR2	Player input shall be processed within one frame.	High
NFR3	Controls shall be consistent across all levels.	Medium
NFR4	New levels can be added without changing core logic.	Medium
NFR5	Code shall be modular and maintainable.	Medium
NFR6	The game shall not crash with invalid level data.	High
NFR7	The game shall run on Windows as an executable	High
NFR8	All game transitions are consistent and function properly.	High

Justification of Requirements

Player Control & Mechanics

FR1 – FR10

These requirements are needed to ensure that the user has a way to control the player in order to manoeuvre a stealth environment. To satisfy my stakeholder requirements (Stakeholder 3), mechanics need to be intuitive and engaging, therefore a 360-degree movement system will be implemented. This makes gameplay smoother and enables the user to more easily evade enemies,

Commented [AG1]: Doing this also makes the issue of enemies not having to stick to 8 directional movement when pathfinding, removing the need for a grid based pathfinding system, which is expensive. Use this for algorithm design justification

Interactable objects such as doorframes and hiding spots enable the user to escape certain detection, at a cost of making noise. This is necessary to create engaging gameplay, a cornerstone of my stakeholder requirements.

Creating restrictions to player movement when in special situations is necessary to reduce unintended exploitation of a mechanic. Restrictions are a preventative measure for unintended exploitation, and essential for balancing interactions so that they are not overpowered and overused. This satisfies Stakeholder 2's requirements of a strategy-heavy game, where decisions made should have a plan behind them, and Stakeholder 1's requirements for creating balanced gameplay.

The requirement for a player to have multiple states is more for the design of algorithms used in player logic – a finite state machine can be used to model the player in different situations, and conditional logic can be applied in these situations. Additionally, it can be used as a usability feature – users can clearly see what state the player is in through visual feedback.

Enemy Detection Systems

FR11-FR14

An engaging detection system, like in MGS uses both noise and visibility as variables for detection. Therefore my requirements include detection through noise, which punishes the player for moving too quickly, interacting with "loud" objects, and making contact with an enemy. This satisfies Stakeholder 1's requirements for creating balanced gameplay, where actions are punished by increasing enemy detection.

Visibility is also used for detection, with a steeper escalation of enemy states. This reflects the consequence of making direct contact with an enemy, going from patrolling to chasing the player. Visual confirmation must be established, where the player should be in the enemy's line of sight for at least 1 second before enemy state is altered. This is to balance gameplay and not make enemy AI too frustrating for the user.

Enemy Behaviour & AI States

FR15 – FR21

Enemy Behaviour and AI closely models a real-life stealth scenario, namely how with increasing suspicion, action follows. Enemy patrolling occurs when enemy suspicion is zero, and increases with noise heard. When suspicion is not zero, enemies become suspicious and search the nearby area. If they find clues that result in an exact direction being pinpointed, they search that location. If they make contact with the player, they chase and attempt to kill the player.

This is important to clearly convey enemy states to the player, through observation of enemy behaviour, to inform future decisions made in gameplay. Additionally, it complements a noise and visibility detection system, with conditional logic based on those systems to inform behaviour, such as chasing a player if contact is made, or returning to patrol if no further contact or clues are found.

Level Management & Structure

FR22 – FR25

Level management is important especially with multiple levels to maintain the consistency of a level, meaning enemy positions, walls and interactables need to stay static, so they will be stored in external files that cannot be changed further by the program when distributed. Levels need to be tracked for completion internally, to allow access to further levels on completion as conditional logic, ensuring game progress is consistent and fair.

Global Game State Management

FR26 – FR29

Game state management is necessary in order to control a gameloop. The gameloop will constantly update frames with game logic, so it is necessary for a single condition to be able to stop all those updates and replace them with some other updates matching the scenario. E.g. if a player dies the game should stop running and a pop-up with a "You Died" message should show up. Global game state management allow responsive transitions between dynamic game states to happen.

User Interface & Menus

FR30 – FR33

Menus are necessary for the user to interact with the game. Menus will have buttons and text which describes their function, as well as icons to signify common functions, such as "X" which closes the game. The start menu will be the central hub for the game, where the player can enter settings, play the game and exit the game, and is the initial state of the game. This is necessary to allow the user to intuitively access other menus. Settings menus are necessary for the user to be able to change variables within the program without actually altering code in the scripts. This creates a layer of safety and program integrity. Pop up menus such as pause menus, level completion and failure menus, which don't completely overwrite the screen, are necessary as a visual indicator of being temporary and escapable. They contain the essential buttons for navigation and game function, important for user experience. This satisfies Stakeholder 4's requirements for an intuitive user interface system.

Performance & Quality Constraints (Non-Functional)

NFR1 – NFR7`

Maintaining strong performance (e.g. a stable frame rate and responsive input) is essential to ensure smooth gameplay. In a real-time stealth game, delayed input or frame drops can cause the player to fail due to factors outside their control, which would be unfair. Setting a clear performance requirement ensures that the game behave consistently across different systems and that the game feels responsive during play.

The project will be developed iteratively, with features and content added over time. A requirement that allows new levels, mechanics, or entities to be added without altering core game logic ensures the project can be expanded efficiently as development progresses. This reduces development time, minimises the risk of introducing bugs when adding new content, and allows the game to scale in complexity without requiring major rewrites.

Maintaining clean modular code improves readability, making the project easier to debug, test and extend. By separating responsibilities into modules and classes, individual components can be developed and modified independently. This is necessary in a large programming project as it reduces technical debt and allows faults to be isolated and resolved more quickly.

Crash prevention is essential to ensure the game behaves reliably under both normal and unexpected conditions, such as missing files or invalid data. Implementing defensive programming techniques prevents the program from terminating unexpectedly, which would negatively impact UX and make testing difficult. This requirement increases the overall robustness of the software and ensures that errors fail gracefully rather than crashing the game.

Hardware and Software Configuration

The hardware configuration needed to run this system includes a modern processor, at least 4GB RAM. Inputs from a mouse and keyboard are read by the system, therefore the user will need a mouse and keyboard to play the game. The user will need a monitor, with at least a 30 Hz refresh rate.

The software configuration needed to run this system includes an OS compatible with .exe files, as the game will be distributed as a windows executable. The game should be compatible with versions of Windows 10 and above. For testing remotely and running the scripts for the game themselves, Python 3.11 or above will be needed. Certain python modules will be required as well, such as Pygame, math, etc. This will be determined in design, and an addendum will be added.

Commented [AG2]: Determine complete module list, add software config addendum in design.

Success Criteria

SC ID	Linked FR	Success Criterion	Test Method	Pass Condition
SC1.1	FR1	Can the player move in all directions?	Observe player movement in all directions during gameplay	Player moves in all directions correctly
SC1.2	FR1	Can the player change between movement types walking, running and sneaking?	Observe changes in speed when toggling the change between movement types	Player can move at three distinct speeds through changing their movement type
SC1.3	FR1	For each movement type, does the player move at a consistent speed regardless of direction	Measure euclidean distance over time for multiple directions of movement	Speed variable within acceptable limits.
SC1.4	FR1	Does the player stop when the user stops input?	Initiate movement with input and stop providing input	Player stops when input is stopped.
SC1.5	FR1	Does the player walk by default?	Load game and start moving, without changing movement type	Player speed is the walking speed
SC2.1	FR2	Does the player's walking speed use FPS to calculate the amount of pixels it moves by?	Change FPS up to 60, observe player speed	Player speed is the same across different FPS
SC3.1	FR3	Does the player rotate towards their direction of motion?	Change direction of movement, observe player sprite	Player sprite faces direction of movement
SC4.1	FR4	Is the player restricted to interaction with an object when they are too far?	Change distance from interactable, observe interaction	Player only interacts with object within range
SC4.2	FR4	Does the interaction occur?	Try interaction, observe interactable	Interaction occurs
SC4.3	FR4	Does the interaction stop when player moves out of range	Try interaction, move out of range, observe interactable	Interaction occurs, stops when out of range.

SC5.1	FR5	Can player pick up dead bodies?	Move to dead body, try pick up, move away, observe dead body position	Dead body moves with player.
SC5.2	FR5	Can player drop dead bodies?	Pick up dead body move away, attempt to drop dead body, move away, observe dead body position	Dead body stays where player dropped it.
SC5.3	FR5	Does dropping dead body near enemy prompt detection	Drop dead body near enemy, observe detection	Enemy should detect noise.
SC5.4	FR5	Can body be hidden in hiding spot?	Pick up dead body, interact with hiding spot	Dead body should go in hiding spot.
SC6.1	FR6	Does running produce noise?	Run near enemy without LoS, observe detection	Enemy should detect noise.
SC6.2	FR6	Does walking produce noise?	Walk near enemy without LoS, observe detection	Enemy should detect noise, less audibly than running
SC6.3	FR6	Does attacking produce noise?	Attack an enemy with LoS near another enemy without LoS, observe detection	Enemy should detect noise
SC6.4	FR6	Do interactions with a door produce noise?	Interact with a doorframe near an enemy, observe detection	Enemy should detect noise
SC6.5	FR6	Do interactions with hiding spots produce noise	Interact with a hiding spot near an enemy, observe detection	Enemy should not detect any noise
SC6.6	FR6	Does sneaking produce noise?	Sneak near enemy without LoS, observe detection	Enemy should not detect any noise
SC7.1	FR7	Can the player move through walls?	Move player into wall, observe collision	Player momentum stops before wall
SC7.2	FR7	Can the player move through uninteracted doors?	Move player into uninteracted door, observe collision	Player stops before uninteracted door
SC7.3	FR7	Can the player move through interacted doors?	Move player into interacted door, observe collision	Player should pass through door.

SC7.4	FR7	Can the player move through obstacles?	Move player into obstacle, observe collision	Player stops before obstacle.
SC8.1	FR8	Can player run when carrying dead body?	Collect dead bodty, attempt to run	Player cannot run
SC8.2	FR8	Does player regain the ability to run when dropping dead body?	Collect dead body, drop dead body, attempt to run	Player can run
SC8.3	FR8	Can the player move when hidden?	Interact with hiding spot, attempt to move	Player cannot move.
SC8.4	FR8	Can player hide in hiding spot with dead body?	Interact with hiding spot with dead body, observe interaction	Player picks up dead body, player is not hidden.
SC9.1	FR9	Does the player change state to "dead" when health = 0?	Make health = 0, observe player state	Player stops moving, "you died" message shows up, gameloop ends
SC9.2	FR9	Does the player state change to hidden when in hiding spot?	Interact with hiding spot, observe player state	Player state is "hidden", player cannot move, player appears less visible
SC9.3	FR9	Does the player state change to normal when leaving hiding spot?	Interact with hiding spot, leave hiding spot, observe player state	Player regains movement, player state = "normal", player is fully visible.
SC9.4	FR9	Does player state change to "restricted" when carrying dead body	Pick up dead body, observe player state	Player cannot run.
SC10.1	FR10	Is player visible when in "hidden" state?	Be in hiding spot with LoS to an enemy patrol, observe detection	Enemy cannot see player
SC10.2	FR10	Is player visible when in "normal" state?	Be in LoS to an enemy, observe detection	Enemy can see player
SC10.3	FR10	Is player visible when in "restricted" state?	Be in LoS to an enemy, observe detection	Enemy can see player

SC11.1	FR11	Can enemy hear noise above a audible threshold intensity?	Generate a noise with intensity equal to the threshold and above it, observe detection	Enemy can hear noise
SC11.2	FR11	Can enemy hear noise below the audible threshold intensity	Generate a noise with intensity below the threshold, observe detection	Enemy cannot hear noise
SC11.3	FR11	Can enemy determine source of noise above the directional audible threshold intensity?	Generate a noise with intensity equal to the threshold and above it, observe detection	Enemy can hear noise and determine source of noise
SC11.4	FR11	Can enemy determine source of noise below the directional audible threshold intensity?	Generate a noise with intensity between audible and direction audible thresholds, observe detection	Enemy can hear noise but not determine detection.
SC11.5	FR11	Does the intensity of noise generated decay with distance from the source.	Generate a noise, measure intensity at different distances from source	Decrease observed as distance increases.
SC11.6	FR11	Do enemies hear other enemy attacks?	Generate enemy attacks, observe surrounding enemy detection	Enemies within directional audio threshold set last player location to enemy location.
SC12.1	FR12	Does running produce more noise than walking?	Generate running noise and walking noise equidistant to an enemy, measure intensity of respective noises heard	Intensity of running noise is always higher than walking
SC12.2	FR12	Does enemy shooting produce a noise that can be heard across the whole level?	Generate enemy shooting noise, observe enemy reactions across room	All enemies must be able to hear the noise, some may hear source of noise.

SC12.3	FR12	Does dropping dead body create enough noise for nearby enemies to detect where it has been dropped?	Generate body dropping noise near enemies, observe reaction	Nearby enemies will hear noise and determine its source's direction
SC12.4	FR12	Does using a hiding spot generate enough noise for nearby enemies to hear but not direct?	Generate hiding spot interaction noise, observe detection	Nearby enemies will hear noise but not determine direction
SC13.1	FR13	Can enemy detect player with direct line of sight	Establish LoS with enemy, observe detection	Enemy can see player
SC13.2	FR13	Can enemy detect hidden player with direct line of sight to hiding spot?	Establish LoS while hidden with enemy, observe detection	Enemy cannot see player
SC13.3	FR13	Can enemy detect dead body with direct LoS	Establish LoS between dead body and enemy, observe detection	Enemy can see dead body
SC13.4	FR13	Can enemy detect hidden dead body with direct Los	Establish LoS between hidden dead body and enemy, observe detection	Enemy cannot see dead body
SC13.5	FR13	Can enemy detect other enemies with LoS	Establish LoS between two enemies, observe detection	Enemy can see other enemy
SC14.1	FR14	Can enemy see player through wall	Establish LoS between player and enemy with a wall in between, observe detection	Enemy cannot see player
SC14.2	FR14	Can enemy see player through obstacle	Establish LoS between player and enemy with a obstacle in between, observe detection	Enemy cannot see player
SC14.3	FR14	Can enemy see player through closed door	Establish LoS between player and enemy with a closed door in between, observe detection	Enemy cannot see player
SC14.4	FR14	Can enemy see player through open door	Establish LoS between player and enemy with a open door in between, observe detection	Enemy can see player

SC15.1	FR15	Does enemy patrol along a predetermined set of points?	Record predetermined set of points, observe change enemy position in "patrol" state	Enemy travels along points in order from first to last, going back to the first in a circular fashion.
SC15.2	FR15	Are patrol points circular in nature?	Record enemy path in "patrol" state between last point and first point	Enemy does not cross other points within path to get to the first one.
SC15.3	FR15	Do patrol paths sufficiently enable the enemy to observe the whole room within one patrol interation	Record enemy in "patrol" state, observe enemy vision cone	Majority (<90%) of the room should be observed in one patrol iteration.
SC15.4	FR15	Do patrol paths account for walls?	Observe enemy in "patrol" state moving between two points obstructed by a wall	Enemy finds a path around walls, not passing through them.
SC15.5	FR15	Do patrol paths account for door objects?	Observe enemy in "patrol" state moving between two points obstructed by a door	Enemy should open door before walking through
SC15.6	FR15	Do patrol paths account for obstacles?	Observe enemy in "patrol" state moving between two points obstructed by an obstacle	Enemy finds a path around obstacles, not passing through them
SC16.1	FR16	Does enemy state change from "patrol" to "search" when enemy has directional audio cue?	Generate directional audio cue near enemy, observe enemy state change	Enemy state should change from "patrol" to "search"
SC16.2	FR16	Does enemy pathfind to last player location when in "search"?	Observe enemy in "search" state, record last player location, observe enemy movement and position	Enemy should find the most optimal path to the last player location and follow it.

SC16.3	FR16	Does enemy cancel search when there is no new last player location?	Observe enemy in "search" state at the last player location, observe enemy state change	Enemy should change back to "patrol" state
SC16.4	FR16	Does enemy update last player location when in "search" state	Observe enemy in "search" state, generate directional audio cue, observe last player location	Last player location should update to the location of the new audio cue.
SC16.5	FR16	Does enemy state change from "chase" to "search" when losing LoS to player during chase	Observe enemy in "chase" state, remove LoS between enemy and player, observe state change	Enemy state changes from "chase" to "search"
SC16.6	FR16	Does enemy change to "chase" state from "search" state when establishing LoS between player and enemy?	Observe enemy in "search" state, establish LoS between player and enemy, observe enemy state change	Enemy state changes from "search" to "chase"
SC16.7	FR16	Does last player location reset to Null when enemy reaches last player location without any more directional audio cues or LoS?	Observe transition from "search" to "patrol", record last player location	Last player location = Null
SC17.1	FR17	Does enemy pathfind to nearest patrol point after transitioning back from "search"?	Observe transition from "search" to "patrol", record nearest point through euclidean distance, observe enemy position	Enemy should reach the nearest patrol point.
SC17.2	FR17	Does enemy continue patrolling when concluding search?	Observe enemy after transition from "search" to "patrol", record path taken	Enemy path should match predetermined patrol path
SC18.1	FR18	Does enemy transition from any "alive" state to "chase" when LoS between enemy and player is established?	Observe enemy state after establishing LoS between player and enemy	Enemy state should change to "chase"

SC18.2	FR18	Does enemy follow the player when in "chase" state?	Observe enemy in "chase" state, observe enemy movement and player movement	Enemy follows player
SC18.3	FR18	Does enemy maintain a minimum distance of 50 pixels from player?	Observe distance between enemy in "chase" state and player, let enemy shorten distance to player	Enemy should close the distance until it is 50 pixels away from the player.
SC18.4	FR18	Does enemy transition to "chase" state when LoS is established between enemy and another enemy in "chase" state	Observe enemy with LoS to another enemy in "chase" state, record enemy state	Enemy transitions to "chase" state, following enemy
SC18.5	FR18	Does an enemy transition to "patrol" when following another enemy who also transitions to "patrol"?	Observe enemy with LoS to another enemy in "chase" state, subsequently change the other enemies state to "patrol" and observe the first enemy's state.	First enemy's state should also change to "patrol"
SC19.1	FR19	Does enemy attack the player within 300 pixels of the player?	Observe distance between enemy in "chase" state and player, observe enemy attack	Enemy attacks player when distance < 300
SC19.2	FR19	Does enemy attack faster when stationary in chase state: distance < 100	Observe enemy attack when distance between enemy in "chase" state and player is 100 pixels or less	Enemy stops, attacks much faster and more accurately
SC19.3	FR19	Are enemy attacks accurate?	Observe multiple enemy attacks on stationary player, record accuracy, repeat for multiple distances.	Enemy shoots stationary player with 50%<x<60% accuracy from a distance of 300 pixels, increasing as distance decreases

SC19.4	FR19	Does enemy slow down when shooting at player?	Observe enemy in "chase" state, record speeds when attacking and when not attacking player	Speed when attacking player is lower than speed when not attacking player
SC19.5	FR19	Are enemy attack times predictable?	Observe enemy in "chase" state, record timings between attacking and chasing periods	Timings are completely random, occuring often.
SC19.6	FR19	Do enemy attacks make noise?	Observe noise generated from enemy attacks	Noise is generated from enemy attacks
SC20.1	FR20	Do successful enemy attacks damage the player?	Observe successful attack, record player health	On successful attack, player health reduces.
SC22.1	FR22	Can more than one distinct level be loaded and played?	Load at least two different level files	Both levels function as intended and are distinct
SC23.1	FR23	Can level data be stored on external files?	Generate an external file for a level, attempt to load from the external file	Level data is extracted from file and loads on screen
SC23.2	FR23	Does modifying a level file change the level in-game without code changes?	Edit a level file (e.g. enemy count or obstacle position) and reload the game.	
SC24.1	FR24	Do level files store enemy positions and patrols	The game reflects the modified level data without altering source code.	Enemy positions and patrols load correctly
SC24.2	FR24	Do level files store the layout (obstacles, walls, interactables) of a level?	Load level from external file	Level layout loads with correct walls, obstacles, hiding spots and doorframes

SC25.1	FR25	Does the level enter an active state when gameplay begins	Load level, observe level state	Level activity becomes true
SC25.2	FR25	Does level state change to "completed" when level is completed?	Load level and complete the level, observe level state	Level state = completed
SC25.3	FR25	Does level state default to incomplete when level has not be completed?	Record starting state of level, load and fail level and record its state	starting level state = incomplete, level state after failed playthrough = incomplete.
SC26.1	FR26	Does the game enter the play state when a new game is started from the start menu?	Initiate “Start Game” from the start menu and observe the game state	Game state changes to play and player control is enabled
SC26.2	FR26	Is the play state active immediately after a level loads?	Start a level and check whether gameplay logic (movement, enemies) is running	Player and enemies update every frame
SC26.3	FR26	Are non-play states inactive while the game is in play state?	Start gameplay and monitor pause/menu logic execution	Only play-state logic executes
SC27.1	FR27	Does pressing the pause input switch the game to the pause state?	Press the pause key during gameplay	Game state changes to pause
SC27.2	FR27	Is gameplay logic halted while the game is paused?	Pause the game and observe player and enemy movement	No movement or AI updates occur
SC27.3	FR27	Can the pause input only pause the game during play state?	Press the pause key in menus or completion state	Game does not enter pause state
SC28.1	FR28	Does the game detect when the success condition is met?	Complete the level objective	Success condition is detected
SC28.2	FR28	Does the game transition to the completion state on success?	Complete the level and observe the game state	Game state changes to completion

SC28.3	FR28	Is gameplay logic disabled in the completion state?	Attempt to move or interact after completion	No gameplay interactions occur
SC29.1	FR29	Does the game detect when the player is killed?	Allow an enemy to kill the player	Player death condition is triggered
SC29.2	FR29	Does the game enter the dead state when the player is killed?	Observe the game state after death	Game state changes to dead
SC29.3	FR29	Is player input disabled in the dead state?	Attempt to move or perform actions after death	No player input is processed
SC30.1	FR30	Does the start menu provide an option to start the game?	Open the start menu and inspect available options	Start option is present
SC30.2	FR30	Does selecting “Start” begin gameplay?	Select the start option	Game enters play state
SC30.3	FR30	Does the start menu provide an option to exit the game?	Select the exit option from the start menu	Game closes successfully
SC31.1	FR31	Does the pause menu appear when the game is paused?	Pause the game during gameplay	Pause menu UI is displayed
SC31.2	FR31	Does selecting “Resume” return the game to the play state?	Pause the game and select resume	Game state returns to play
SC31.3	FR31	Does selecting “Quit” exit to the start menu or close the game?	Select quit from the pause menu	Game exits as designed
SC32.1	FR32	Can at least one setting be modified in the settings menu?	Open settings and modify a setting	Setting value changes
SC32.2	FR32	Are modified settings applied correctly?	Change a setting and observe its effect	Setting behaviour updates correctly
SC32.3	FR32	Do modified settings persist after exiting the settings menu?	Change settings, exit menu, and re-open	Modified settings remain applied

SC33.1	FR33	Does the UI update when the game state changes?	Transition between play, pause, and menu states	Correct UI is displayed for each state
SC33.2	FR33	Is gameplay UI hidden when not in play state?	Enter pause, completion, or dead states	Gameplay HUD is hidden
SC33.3	FR33	Does the UI reflect completion and death states correctly?	Complete a level and die in separate tests	Correct completion or death UI is displayed
SCNFR1.1	NFR1	Does the game maintain at least 60 FPS during normal gameplay?	Run the game and monitor FPS during active gameplay	FPS remains at or above 60
SCNFR1.2	NFR1	Does the game maintain 60 FPS during high-load scenarios?	Stress test with multiple enemies and effects	FPS does not drop below 60
SCNFR2.1	NFR2	Is player input processed within a single frame?	Press movement/action keys and observe response timing	Player responds within one frame
SCNFR2.2	NFR2	Does input latency remain consistent during gameplay?	Repeatedly perform inputs under different game states	No noticeable input delay occurs
SCNFR3.1	NFR3	Are control mappings identical across all levels?	Compare control behaviour across multiple levels	Controls behave identically
SCNFR3.2	NFR3	Do player actions trigger the same responses in every level?	Perform identical actions in different levels	Actions produce consistent results
SCNFR4.1	NFR4	Can a new level be added without modifying core game logic?	Add a new level file without changing source code	New level loads successfully
SCNFR4.2	NFR4	Are levels loaded using external data files?	Inspect level loading implementation	Levels are loaded from external files
SCNFR5.1	NFR5	Is the codebase organised into logical modules?	Review code structure and file organisation	Code is separated into distinct modules

SCNFR5.2	NFR5	Can individual modules be modified without affecting unrelated systems?	Modify a module and run regression tests	Unrelated systems remain unaffected
SCNFR6.1	NFR6	Does the game handle invalid level data without crashing?	Load a level with missing or incorrect data	Game continues running
SCNFR6.2	NFR6	Are errors from invalid data handled gracefully?	Observe behaviour when invalid data is loaded	Error is handled without termination
SCNFR7.1	NFR7	Does the game run correctly on Windows?	Run the game on a Windows system	Game launches and runs
SCNFR7.2	NFR7	Can the game run without python installed on another computer with Windows OS?	Run game on another computer without Python, as an executable file.	Game runs correctly, with same functionality as running on Python with dependencies
SCNFR8.1	NFR8	Do all game state transitions function correctly?	Trigger all state transitions during gameplay	No incorrect or missing transitions occur
SCNFR8.2	NFR8	Are game transitions consistent across multiple play sessions?	Repeat transitions in multiple runs	Transitions behave consistently

Approach to Design and Development

To aid my design and development stages of my project, I will be using Jira to track tasks, manage milestones and store development logs. This will allow me to organise my work systematically, prioritise requirements and monitor progress throughout the project. Using Jira also helps ensure that no part of the project is overlooked and makes it easier to adapt my plan if challenges arise.

For version control, I will be using GitHub, which will allow me to maintain a history of all code changes, safely experiment with new features, and revert to previous versions if necessary, reducing the risk of losing progress.

My development approach will follow an iterative methodology, where I plan, implement, test and refine features in cycles. This approach allows me to focus on building a functional prototype early and gradually add complexity while ensuring that each component works correctly.

Design

Top Level Decomposition

The development of the solution will take a modular, object-oriented approach, where the overall system is decomposed into smaller, independent components. Each major entity and system within the game is represented as a separate module or class, such as the Player, Enemy, Game States, UI, etc.

This approach allows each component to be designed, implemented and tested independently, reducing overall complexity and making the codebase easier to understand and maintain. For example, enemy behaviour can be modified or extended without affecting player controls or level loading logic.

Decomposition also supports clear separation of responsibilities which is important for managing game states such as playing, paused, level completion, and game over. By

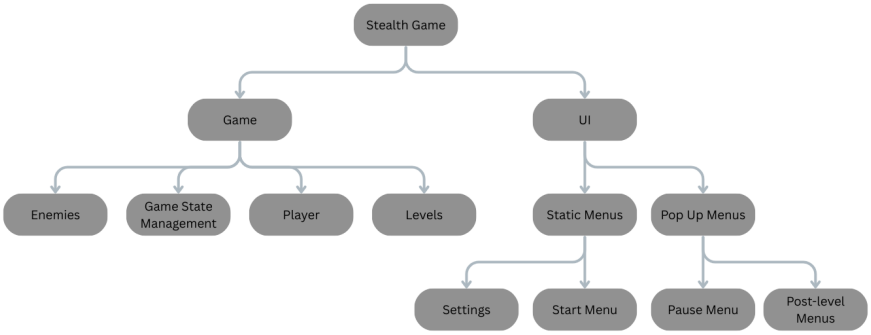
isolating state logic into modules, transitions between states can be controlled consistently.

Using object-oriented techniques such as encapsulation and inheritance further improves scalability. New enemy types or levels can be added by reusing and extending existing classes rather than rewriting logic, reducing duplication and development time.

Overall, decomposition improves maintainability, testability and extensibility, which ensures the game can evolve through development without requiring significant changes to the core framework.

Object oriented approach to decomposition

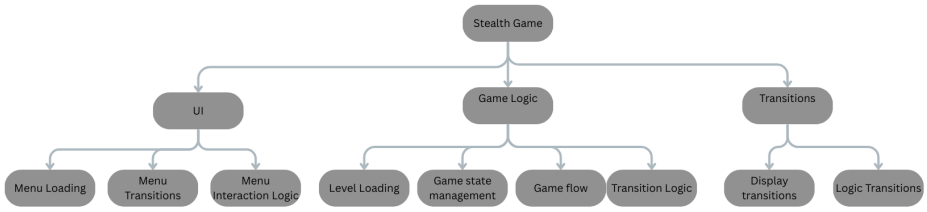
This decomposition version uses an object-oriented approach:



This top-level decomposition diagram splits the game into two major components, User Interface and Game logic. Game logic contains the main objects: enemies, the player, and levels. It also contains a module for game state management, which is a major subsystem to control the flow of the game. The UI section contains objects for Static and Pop-up Menus, both of which are further decomposed into the different branches of menus, which are inherited from their parent classes. The intermediate section for transitions between UI and Game states have been omitted from this TLD, but will be referenced in future design sections.

Modular approach to decomposition

This decomposition version uses a more modular approach:



This diagram presents a modular decomposition of the stealth game, focusing on functional responsibilities rather than individual objects. The system is divided into three major modules: Game, UI and Transitions.

The game module is responsible for core gameplay behaviour, including level loading, game flow and transition logic related to gameplay states. This ensures that gameplay mechanics operate independently of how they are presented to the player.

The UI module handles menu loading, menu interactions and menu-specific trnasitions. This separation allows UI elements to be modified or extended without impacting core gameplay logic.

The Transitions module acts as an intermediary between the Game and UI, managing both logical transitions and display transitions. Treating transitions as a separate module improves clarity, maintainability and consistency across the system.

This modular structure reduces coupling between systems and improves scalability.

Evaluating the models

Both models have their advantages and disadvantages. The object-oriented approach decomposed the system into distinct entities such as the player, enemies, levels and menus. Each entity encapsulates its own data and behaviour, allowing responsibilities to be clearly defined. This also reduces duplications through inheritance and reuse.

This approach maps directly to the implementation language, Python, and supports extensibility, for example, new enemy types or levels can be added with minimal changes to existing code. However, this approach does not attempt to explain how these classes function together and co-exist.

The modular approach focuses on separating the system by functional responsibility, such as game logic, UI handling and transition management. By isolating transitions and flow control into modules, this approach provides a clear overview of how different moving parts of the system interact during execution.

However, it is more abstract and does not map to individual classes, whereas the object-oriented approach provides a clearer structure for implementation, and functionality can be localised to a specific class rather than being generalised for all object types in a module. The object-oriented approach gives flexibility for variation in certain functions and algorithms through inheritance, whereas the modular approach will require new functions to be designed for specific objects, increasing development time.

The final design will adopt an object-oriented implementation, informed by the modular architectural model for informing how objects connect and co-exist. This combined approach ensures that the system remains maintainable, scalable and logically structured, while also allowing clear control over game flow and state transitions.

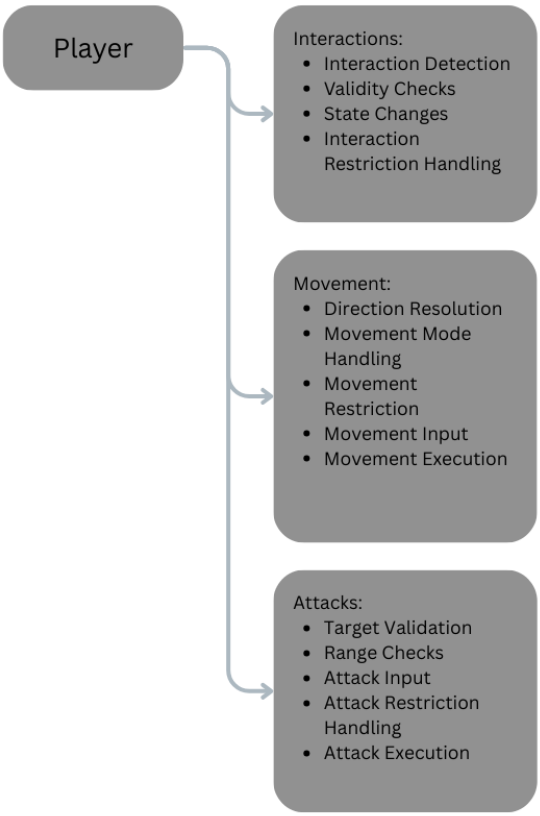
Commented [AG3]: With OOP, hard to understand timing in terms of updates, etc. Need to clarify in key algorithms of how gameloop works and objects are updated in realtime

Game Decomposition

This section looks at the decomposition of the classes of the preferred object-oriented decomposition model, where each class is decomposed into their core functions.

Player Decomposition

The following diagram outlines the decomposition for the player class:



The Player class is decomposed into three main groups of functions: interactions, movement, and attacks. These groups reflect distinct categories of behaviour and allow related logic to be organised cohesively while maintaining clear responsibility boundaries.

The movement group identifies the key functions required to implement a complete movement system, including direction resolution, movement mode handling, input processing, restriction checks, and execution. Separating these concerns ensures that changes to movement mechanics do not affect attack or interaction logic.

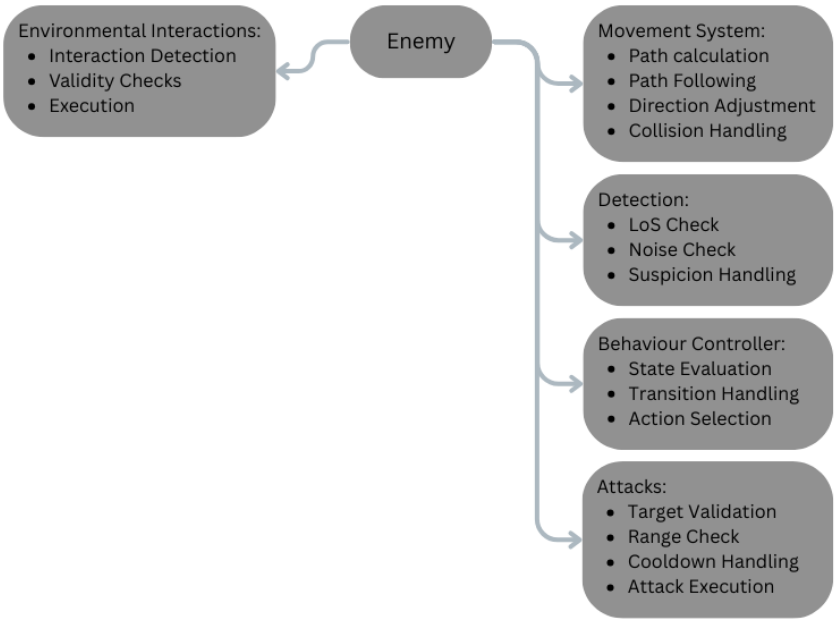
The attacks group contains the functions required to validate and execute combat actions. This includes target validation, range checking, restriction handling, and execution. Attacks are separated from other interactions because they involve dynamic entities with independent behaviour. Enemies are not passive interactables; they possess their own state systems and reactions. Grouping attacks with general interactions would require abstraction that could obscure the complexity of combat logic and make the design of attack algorithms less clear.

The interactions group manages the player's interaction with static or semi-static environmental elements. These interactions generally follow a binary structure: a condition is checked, and if valid, the interaction is executed. Because these interactions share a similar logical pattern, grouping them allows for reusable validation and execution logic. This reduces duplication and improves maintainability.

The distinction between attacks and other interactions is further justified by inheritance structure. Environmental interactables can inherit from a shared base class, whereas enemies cannot be treated purely as interactables due to their broader behavioural systems. Maintaining a separate attack handler preserves clarity in the class hierarchy and supports extensibility.

Enemy Decomposition

The following diagram outlines the decomposition for the enemy class

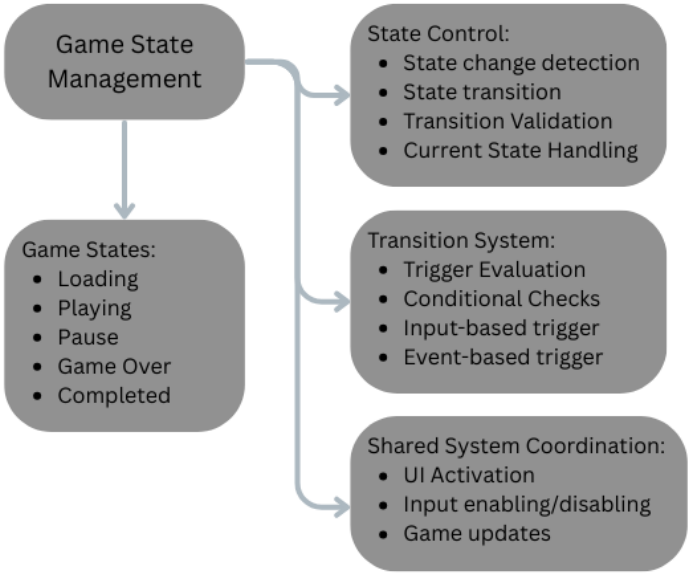


The enemy class is decomposed into five main groups: Movement, Detection, Behaviour Control, Attacks and Environmental Interaction. The detection sub-class is a representation of how the enemy takes in inputs, to inform decisions it makes. The behaviour control sub-class takes the inputs from detection, and changes the local enemy state based on this information. This information is used to provide behavioural outputs by changing the variables that inform the two output sub-classes, movement and attacks, which depend on the local state of the enemy. The environmental interaction section is a static sub-class, which remains the same regardless of enemy state, performing interactions based on proximity/enemy movement.

This decomposition follows a Sense-Think-Act agent architecture, separating perception, decision-making and action into distinct modules. This clearly separates enemy inputs from their outputs, which makes the system easier to reason about, extend, and maintain, while preventing decision logic from becoming tightly coupled with execution logic.

Game State Management Decomposition

The following diagram outlines the decomposition for the Game State Management System:



The Game State Management class is composed of 4 groups: State Control, Transition System, Shared System Coordination and Game States. The state control group represents how the game changes state depending on conditions set by the gameloop. It handles how states transition, and verifies the transition's validity. This is separated from other systems as it acts as a control unit for multiple actions that stem from a transition.

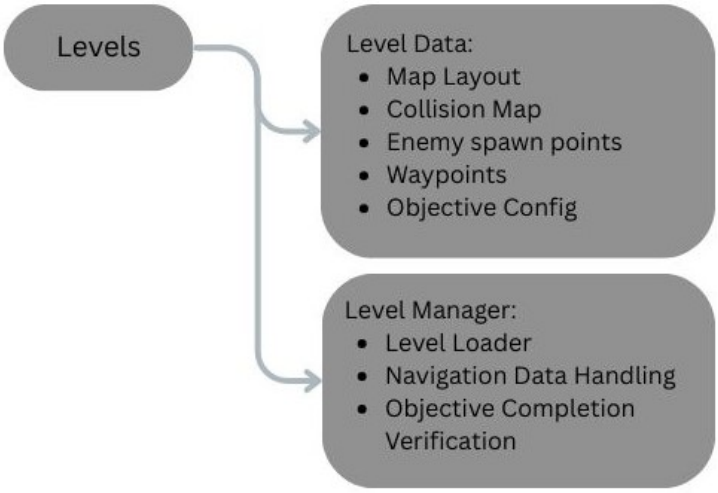
The transition system checks conditions from the gameloop and evaluates whether any event/input triggers a transition, then returns the transition state to the State Controller. This is separated from other systems as its runtime is different in the gameloop, where states are checked every frame. The system has to be minimalistic; only checking for any state changes through conditional logic.

The shared system coordination system handles how the gameloop interacts with separate systems such as the UI, Input handling, etc. For specific actions that lead to a UI menu being loaded, the correct processes need to suspend/restart in order for the game to function properly. This subsystem handles the logic that spans across multiple systems as a result of transitions. This is separated from the State Control, as the state control handles local transitions within the game, whereas the shared system coordination handles pop-up menus such as the pause menu.

The Game States is not a subsystem, but a representation of the overarching states that the game loop can be in. This is needed to inform my design of the state machine controlling the game loop.

Level Decomposition:

The following diagram outlines the decomposition of the Level Object:



The Level class is responsible for managing all data and logic specific to an individual level. Its responsibilities include storing level config data, loading and initialising entities, managing navigation structures such as waypoints, and verifying completion conditions.

Internally, the class is decomposed into static data configuration and level interactions with the game loop. The level manager subsystem is comprised of both static functions and runtime functions; level loader would be during the transition between UI and gameplay, whereas navigation data handling and objective verification would be processed during the game loop, during runtime.

Waypoint handling is maintained within the Level data subsystem as waypoints are pre-generated depending on the layout of the map, and a graph is provided to the enemy class with waypoints before runtime. This prevents the embedding of map-specific logic inside the enemy class.

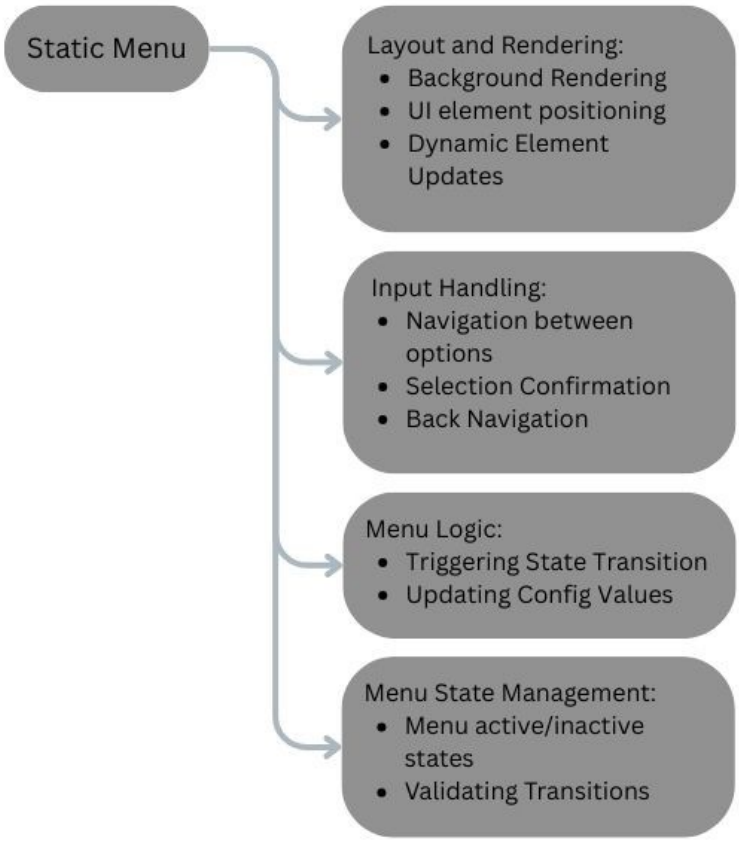
Objective completion verification is centralised within the Level Manager subsystem, where it is handled internally. This avoids distributing completion logic across multiple classes. The level class instead signals Game State Manager when completion criteria are met.

This decomposition ensures clarity of responsibility, supports extensibility for future levels and prevents excessive coupling between gameplay systems

UI Decomposition

Static Menu Decomposition

Static menus are types of UI that replace the gameplay loop. The following diagram outlines the decomposition of Static Menus, mainly their functionality rather than use cases.



The Static Menu class is responsible for the safe loading of static menus such as the start and settings menu. There are four functional groups for this system: Layout and Rendering, Input Handling, Menu Logic and Menu State Management.

Layout and rendering controls the visual aspects of the menu and their rendering during runtime. The UI needs to be positioned correctly in order to allow the user to interact with the UI, and dynamic elements such as buttons need to show visual confirmation of their selection.

Input Handling is a separate subsystem because the user needs to be able to interact with the menu buttons correctly, and each button should perform their designated function. The system needs to constantly read for inputs via mouse with the cursor situated over a certain area for multiple UI elements. The separation is justified as this constantly provides inputs for menu and game transition logic, distinguishing it from output and processing systems.

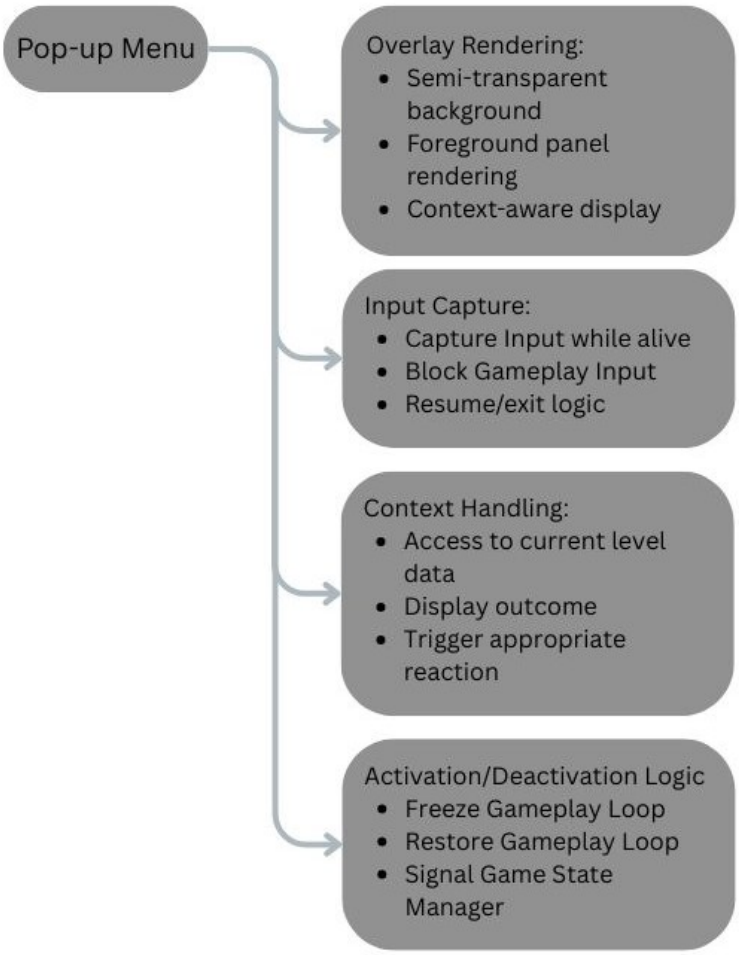
Menu Logic contains the conditional logic necessary to enact a transition between menus and perform the logic behind user interactions such as the changing of settings; the given setting should update.

Menu State management solely controls the states where a static menu is active, and when it is inactive. It controls when the logic behind rendering and input handling is run.

This decomposition ensures clarity of responsibility, whilst being extensible for future menus to be added.

Pop-up Menu Decomposition

The pop-up menu overlays the current gameplay/UI screen to provide temporary functions/act as a user-controllable transition to other game states. The following diagram outlines the decomposition of the Pop-Up Menu Class



The Pop-Up Menu Class is responsible for the safe loading of pop-up menus as a transition/temporary state from a logic-heavy live game loop. It is decomposed into four functional groups: Overlay Rendering, Input Capture, Context Handling and Activation/Deactivation Logic

Overlay rendering is important to ensure that the overlay is visible to the user, and that gameplay appears to be suspended in the background, meaning the user understands that this is a temporary state. This is separated from other logic as it is purely an output system.

Input Capture is necessary to ensure that the user can interact with the UI properly, whilst ensuring that all gameplay inputs are blocked and the user cannot simultaneously interact with the UI and gameplay. The input capture system is separated from other systems within the class as it is purely an input system.

Context Handling is a one-time background process that generates all the level statistics for the user to be able to see, for closure purposes. This is separated from Activation/Deactivation logic as it is a data access system, independent of any inputs or outputs, only the type of pop-up menu.

Activation/Deactivation logic is a coordination system that interacts with the Game Management System, in order to enact the transitions between gameplay and pop-up UIs.

This decomposition ensures clarity of responsibility, allows extensibility with the variety and function of pop-up menus, whilst being an intermediary for the UI and Game states to co-exist.

Class Diagrams

Development

Testing

Evaluation